WDJ - QB UNIT 4 (5- marks)

1 Define Spring Framework and explain IoC.

1. Spring Framework is a popular Java framework used for developing enterprise applications. It helps developers create scalable, secure, and maintainable applications easily.
2. Spring provides many features like dependency injection, security, transaction management, and web development support. It reduces the complexity of Java application development.
3. IoC stands for Inversion of Control. It is a principle where the control of object creation and management is transferred from the programmer to the Spring container.
4. In traditional programming, developers manually create objects using the new keyword. In Spring, the container automatically creates and manages objects called beans.
5. IoC helps in reducing tight coupling between classes. It makes the application more flexible and easier to test.
6. Spring uses configuration files or annotations to manage IoC. The container injects required dependencies into objects automatically.
7. The main advantage of IoC is better code reusability and maintainability. It also improves application development speed and simplifies project management.

2 What is Dependency Injection (DI)? Explain its types.

1. Dependency Injection is a feature of Spring Framework used to provide objects to another object automatically. It helps in reducing dependency between classes.
2. In DI, the Spring container creates objects and injects required dependencies into them. This improves flexibility and makes code easier to maintain.
3. DI supports loose coupling in applications. Loose coupling makes testing and updating applications simpler.
4. The first type of DI is Constructor Injection. In this method, dependencies are provided through the constructor of the class.
5. Constructor Injection ensures that all required dependencies are available during object creation. It is commonly used for mandatory dependencies.
6. The second type of DI is Setter Injection. In this method, dependencies are provided using setter methods after object creation.
7. Setter Injection is useful when dependencies are optional. It gives more flexibility to modify dependencies later if needed.

3 Explain basics of Spring Security.

1. Spring Security is a framework used to secure Java applications. It provides authentication and authorization features for applications.
2. Authentication is the process of verifying the identity of a user. Spring Security checks usernames and passwords before allowing access.
3. Authorization determines what resources a user can access after login. Different users can have different roles and permissions.
4. Spring Security protects applications from common attacks like session fixation and cross-site request forgery. It improves overall application security.
5. It supports form-based login and role-based access control. Developers can define access rules for different pages and functions.
6. Spring Security can be integrated easily with Spring Boot applications. Configuration can be done using Java configuration or annotations.
7. The framework also supports password encryption and secure session management. This helps in protecting sensitive user information.

---

4 What is Spring Boot?

1. Spring Boot is an extension of the Spring Framework used for rapid application development. It simplifies the configuration and setup process of Spring applications.
2. Spring Boot provides pre-configured templates called starter dependencies. These starters help developers quickly add required features to projects.
3. It reduces the need for writing large XML configuration files. Most configurations are done automatically using auto-configuration.
4. Spring Boot includes an embedded server like Tomcat or Jetty. Applications can run directly without deploying them separately on external servers.
5. It supports easy creation of standalone and production-ready applications. Developers can start projects quickly with minimal setup.
6. Spring Boot works well with databases, REST APIs, and microservices. It is widely used for modern web application development.
7. The framework improves developer productivity and reduces development time. It also provides monitoring and management features for applications.

5 Explain evolution from Spring to Spring Boot.

1. Initially, developers used the traditional Spring Framework for enterprise Java applications. It required large XML configurations and complex project setup.
2. In the early stage, Spring applications were difficult for beginners to understand. Developers had to manage dependencies and server configuration manually.
3. Later, annotation-based Spring was introduced to reduce XML usage. Annotations like @Component and @Autowired made configuration simpler.
4. After that, Java-based configuration became popular in Spring applications. Developers could configure applications using Java classes instead of XML files.
5. Spring Boot was introduced to simplify Java development further. It provided automatic configuration and reduced boilerplate code.
6. Spring Boot added features like embedded servers and starter dependencies. Applications could run directly without installing external servers.
7. Today, Spring Boot is widely used for REST APIs, web applications, and microservices. It improves development speed and makes application setup very easy.

6 What are limitations of traditional Spring configuration?

1. Traditional Spring applications required large XML configuration files. Managing these files became difficult in large projects.
2. Developers had to configure dependencies manually in the project. This increased the chances of version conflicts and errors.
3. Setting up a Spring project was time-consuming for beginners. A lot of effort was needed before writing actual business logic.
4. Traditional Spring required external servers like Tomcat for deployment. Developers had to install and configure the server separately.
5. The framework involved complex project structure and configuration management. This slowed down application development speed.
6. Beginners often found Spring difficult because of too many setup steps. Understanding XML configuration and server setup required extra learning.
7. Due to configuration overload, developers spent more time on setup than coding. These limitations led to the development of Spring Boot.

7 Define auto-configuration in Spring Boot.

1. Auto-configuration is a feature of Spring Boot that automatically configures applications. It reduces the need for manual configuration by developers.
2. Spring Boot checks the libraries and dependencies available in the classpath. Based on them, it applies suitable default configurations automatically.
3. This feature helps developers start projects quickly with minimal setup. It saves time and reduces boilerplate code.
4. For example, when spring-boot-starter-web dependency is added, Spring Boot automatically configures Spring MVC and Tomcat server.
5. Auto-configuration also creates important components like DispatcherServlet. Developers do not need to configure these manually.
6. It follows the concept of "convention over configuration." The framework makes common decisions automatically for the developer.
7. Developers can still customize or override the default configuration if required. This provides both simplicity and flexibility in application development.

---

8 What is embedded server in Spring Boot?

1. An embedded server is a built-in web server provided inside a Spring Boot application. It allows applications to run without external server installation.
2. Spring Boot supports embedded servers like Tomcat, Jetty, and Undertow. Tomcat is the default server used in most applications.
3. In traditional Spring applications, developers had to install and configure external servers manually. This process increased complexity and deployment time.
4. With Spring Boot, the application carries its own server internally. Developers can run the application directly as a Java program.
5. Embedded servers simplify deployment and application execution. They make the development process faster and easier.
6. The main class in Spring Boot automatically starts the embedded server during application startup. This reduces manual configuration work.
7. Embedded servers are very useful in microservices and cloud-based applications. They improve portability and make deployment more convenient.

9 Explain opinionated defaults in Spring Boot.

1. Opinionated defaults are predefined configurations provided by Spring Boot. They help developers start applications quickly without much manual setup.
2. Spring Boot automatically chooses common settings that are suitable for most applications. This reduces the need for writing configuration code.
3. For example, Spring Boot uses Tomcat as the default embedded server. It also uses port 8080 as the default application port.
4. Default logging configuration is also enabled automatically in Spring Boot. Developers can run applications without configuring logging separately.
5. The framework follows the principle of "convention over configuration." It makes decisions automatically based on standard development practices.
6. Opinionated defaults reduce development time and simplify project setup. Beginners can easily create applications with minimal configuration.
7. Developers can still modify or override these default settings when required. This provides flexibility while keeping development simple and fast.

---

10 Describe Spring Boot project structure.

1. The Spring Boot project structure organizes application files in a proper manner. It helps developers manage code and resources efficiently.
2. The src/main/java folder contains Java source code of the application. It includes packages like controller, service, repository, and model.
3. The controller package handles HTTP requests and responses in web applications. The service package contains business logic of the application.
4. The repository package manages database operations and data access. The model package defines data structures or entity classes.
5. The main class Application.java acts as the entry point of the application. It starts the embedded server and triggers auto-configuration.
6. The src/main/resources folder stores configuration and static files. It contains application.properties, HTML, CSS, JavaScript, and templates.
7. The pom.xml file contains Maven dependencies and build configuration details. The src/test/java folder is used for unit and integration testing.

11 Steps to create a Spring Boot application using Spring Initializer.

1. Open Spring Tool Suite or Spring Initializer to create a new project. Select “Spring Starter Project” from the file menu.
2. Enter project details like project name, type, packaging, and Java version. Usually Maven and Jar packaging are selected for Spring Boot applications.
3. Add required dependencies while creating the project. For web applications, developers commonly select the Spring Web dependency.
4. After clicking finish, the project structure is generated automatically. Spring Boot creates folders like src/main/java and src/main/resources.
5. Create the main application class using the @SpringBootApplication annotation. This class acts as the starting point of the application.
6. Create controller classes and define request mappings using annotations like @RestController and @GetMapping. These controllers handle browser requests.
7. Run the project as a Spring Boot application using the IDE. The embedded Tomcat server starts automatically, and the application runs successfully.

---

12 What is REST architecture?

1. REST stands for Representational State Transfer. It is an architectural style used for developing web services and APIs.
2. REST architecture allows communication between client and server using HTTP methods. It is commonly used in web and mobile applications.
3. REST follows principles like stateless communication and resource-based interaction. Each request from the client contains complete information.
4. Resources in REST are identified using URLs. Clients can access or modify resources through specific endpoints.
5. REST uses standard HTTP methods like GET, POST, PUT, and DELETE. These methods perform operations like retrieving, creating, updating, and deleting data.
6. REST APIs commonly exchange data in JSON format because it is lightweight and easy to understand. This makes communication faster and simpler.
7. REST architecture is widely used because it is simple, scalable, and platform-independent. It supports easy integration between different systems and applications.

13 List REST principles.

1. REST follows the client-server architecture principle. The client handles the user interface while the server manages data and business logic.
2. REST communication is stateless in nature. Each request from the client contains all required information for processing.
3. REST uses resources identified through unique URLs. Every resource can be accessed using a specific endpoint.
4. REST supports a uniform interface for communication between client and server. Standard HTTP methods are used to perform operations on resources.
5. REST architecture allows data representation in formats like JSON and XML. JSON is mostly preferred because it is lightweight and easy to read.
6. REST supports layered system architecture for better scalability and security. Different layers can work independently without affecting each other.
7. REST services are scalable and platform-independent. They allow communication between different applications and systems easily.

---

14 Explain HTTP methods used in REST.

1. HTTP methods are used in REST APIs to perform operations on resources. They define the type of action requested by the client.
2. The GET method is used to retrieve data from the server. It requests information without modifying existing data.
3. The POST method is used to create new resources on the server. It sends data from the client to the server for storage.
4. The PUT method is used to update existing resources completely. It replaces old data with new updated data.
5. The DELETE method is used to remove resources from the server. It permanently deletes the specified data or record.
6. REST APIs use these methods with URLs called endpoints. Different methods on the same URL can perform different operations.
7. HTTP methods make REST APIs simple and standardized. They help applications communicate efficiently over the web.

15 What is @RestController annotation?

1. @RestController is a Spring Boot annotation used to create RESTful web services. It tells Spring that the class will handle web requests.
2. This annotation combines @Controller and @ResponseBody functionality together. It automatically converts returned data into JSON or XML format.
3. Classes marked with @RestController act as REST API controllers. They process client requests and send responses back to the browser or application.
4. It is commonly used in Spring Boot web applications and REST APIs. Developers can define endpoints inside the controller class.
5. Methods inside the controller use annotations like @GetMapping and @PostMapping. These annotations map HTTP requests to specific methods.
6. @RestController reduces the need for additional configuration in web applications. It simplifies REST API development in Spring Boot.
7. This annotation improves code readability and makes API creation faster. It is widely used in modern microservices and web applications.

---

16 Explain GET and POST mapping annotations.

1. @GetMapping and @PostMapping are Spring Boot annotations used in REST controllers. They map HTTP requests to specific controller methods.
2. @GetMapping is used to handle HTTP GET requests. It is mainly used to retrieve data from the server.
3. In a web application, @GetMapping("/") maps the root URL to a method. When the browser accesses the URL, the mapped method executes.
4. @PostMapping is used to handle HTTP POST requests. It is mainly used to send and store data on the server.
5. POST mapping is commonly used in forms, registration systems, and data submission APIs. It helps create new records in the database.
6. These annotations improve readability by clearly defining request types in the controller. They also reduce the need for complex configuration.
7. GET requests are mainly used for fetching data, while POST requests are used for creating data. Both annotations are important for RESTful web services

17 What is JSON? Why is it used in REST APIs?

1. JSON stands for JavaScript Object Notation. It is a lightweight format used for storing and exchanging data.
2. JSON represents data in key-value pair format. It is easy for humans to read and easy for machines to process.
3. REST APIs commonly use JSON for communication between client and server. Data sent from the server is usually returned in JSON format.
4. JSON is platform-independent and supported by many programming languages. This makes it suitable for web and mobile applications.
5. Compared to XML, JSON is simpler and requires less data space. It improves the speed of data transfer in APIs.
6. In Spring Boot REST APIs, objects are automatically converted into JSON responses. This makes API development faster and easier.
7. JSON is widely used because it is lightweight, flexible, and easy to understand. It helps different systems communicate efficiently through REST services.

---

18 What is Postman? How is it used for API testing?

1. Postman is a popular tool used for testing REST APIs and web services. It provides an easy interface for sending HTTP requests.
2. Developers use Postman to test API methods like GET, POST, PUT, and DELETE. It helps verify whether APIs are working correctly.
3. In Postman, users enter the API URL and select the required HTTP method. Requests can then be sent directly to the server.
4. Postman allows users to send data in formats like JSON while testing APIs. This helps developers test request and response handling.
5. The tool displays server responses such as status codes, headers, and JSON data. Developers can easily identify errors and debug APIs.
6. Postman supports authentication and environment variables for advanced API testing. It is useful for testing both simple and complex applications.
7. API testing with Postman improves development speed and application reliability. It is widely used in REST API development and debugging.

(10 – marks)

1 Explain Spring Framework concepts: IoC and Dependency Injection with examples.

1. Spring Framework is a popular Java framework used for enterprise application development. It simplifies Java programming and supports reusable and maintainable code.
2. IoC stands for Inversion of Control. It is a design principle where the control of object creation and management is handled by the Spring container instead of the programmer.
3. In traditional Java programming, developers create objects using the new keyword manually. In Spring, objects are automatically created and managed by the IoC container.
4. The Spring IoC container creates objects, configures them, and manages their lifecycle. It uses XML configuration, annotations, or Java code to manage beans.
5. Dependency Injection is a feature used to provide dependencies to objects automatically. It helps classes remain loosely coupled and easy to test.
6. Spring supports two main types of Dependency Injection: Constructor Injection and Setter Injection. Constructor Injection is used for mandatory dependencies, while Setter Injection is used for optional dependencies.
7. Example of Constructor Injection is a Car class requiring an Engine object through the constructor. The car cannot work without the engine, so the dependency is mandatory.
8. Example of Setter Injection is a Car class receiving a MusicSystem object through a setter method. The car can still work without the music system, so the dependency is optional.
9. IoC and DI together improve flexibility, code reuse, and maintainability in applications. They also simplify testing and reduce tight coupling between classes.
10. These concepts are the core features of the Spring Framework and are widely used in enterprise applications. They help developers create scalable and efficient Java applications.

2 Describe evolution of Spring Boot and its advantages over traditional Spring.

1. Spring Framework was initially used for enterprise Java application development. Traditional Spring applications required large XML configurations and manual setup.
2. Developers faced difficulties because of configuration overload and dependency management issues. Setting up projects and servers required extra time and effort.
3. The next stage introduced annotation-based Spring configuration. Annotations like @Component and @Autowired reduced the use of XML files.
4. Later, Java-based configuration was introduced in Spring applications. Developers could configure applications using Java classes instead of XML configuration.
5. Spring Boot was introduced to simplify Spring application development further. It provided a production-ready framework with minimal configuration.
6. One major advantage of Spring Boot is auto-configuration. It automatically configures components like Spring MVC and Tomcat based on project dependencies.
7. Spring Boot also provides an embedded server such as Tomcat, Jetty, or Undertow. Developers can run applications directly without installing external servers.
8. Starter dependencies in Spring Boot simplify dependency management. Developers can quickly add required features using starter packages.
9. Spring Boot reduces boilerplate code and speeds up development. It is beginner-friendly and supports rapid application development.
10. Today, Spring Boot is widely used for REST APIs, web applications, and microservices. It improves productivity and makes Java application development much easier.

3. Explain features of Spring Boot: auto-configuration, embedded server, and opinionated defaults.

1. Spring Boot is an extension of the Spring Framework used for rapid application development. It simplifies Java development by reducing configuration work.
2. Auto-configuration is an important feature of Spring Boot. It automatically configures the application based on the dependencies available in the project.
3. Developers do not need to write large XML configuration files manually. Spring Boot automatically sets up components like Spring MVC and database configuration.
4. For example, adding the spring-boot-starter-web dependency automatically configures Tomcat server and DispatcherServlet. This saves development time and effort.
5. Another important feature is the embedded server. Spring Boot provides built-in servers like Tomcat, Jetty, and Undertow inside the application.
6. In traditional Spring applications, developers had to install external servers separately. With Spring Boot, applications can run directly as standalone Java applications.
7. Embedded servers make deployment easier and improve portability of applications. They are widely used in microservices and cloud applications.
8. Spring Boot also provides opinionated defaults for common configurations. The framework automatically selects suitable default settings for developers.
9. Examples of opinionated defaults include Tomcat as the default server and port 8080 as the default application port. Default logging configuration is also enabled automatically.
10. These features reduce boilerplate code, simplify development, and improve productivity. Spring Boot is therefore widely used for modern Java web applications and REST APIs.

4. Explain Spring Boot project structure and steps to create and run an application.

1. Spring Boot projects follow a proper structure that organizes source code, resources, and configuration files. This structure makes application development easier and more manageable.
2. The src/main/java folder contains Java source code of the application. It includes packages such as controller, service, repository, and model.
3. The controller package handles web requests and responses in the application. The service package contains business logic, while the repository package handles database operations.
4. The src/main/resources folder stores configuration and static files. It contains application.properties, HTML, CSS, JavaScript, and template files.
5. The main application class contains the @SpringBootApplication annotation. This class acts as the entry point and starts the embedded server automatically.
6. The pom.xml file contains project dependencies and Maven build configuration. It helps manage required libraries and application packaging.
7. The src/test/java folder is used for unit testing and integration testing. Testing helps improve the reliability and performance of the application.
8. To create a Spring Boot application, developers first open Spring Initializer or Spring Tool Suite. Then they enter project details like project name, packaging type, and Java version.
9. After selecting required dependencies such as Spring Web, the project is generated automatically. Developers then create controller classes and write application logic.
10. The application can be run using “Run As → Spring Boot App” in the IDE. Spring Boot starts the embedded Tomcat server, and the application becomes available in the browser.

5 Explain REST architecture and principles with examples.

1. REST stands for Representational State Transfer. It is an architectural style used for developing web services and APIs.
2. REST architecture allows communication between client and server using HTTP methods. It is widely used in web applications, mobile apps, and microservices.
3. In REST, everything is treated as a resource and identified using a unique URL. For example, /students can represent student data in an application.
4. REST follows the client-server principle where the client handles the user interface and the server manages data and business logic. This separation improves scalability and maintainability.
5. Another important REST principle is stateless communication. Each client request contains complete information, and the server does not store client session data.
6. REST also uses a uniform interface for communication between systems. Standard HTTP methods like GET, POST, PUT, and DELETE are used for operations on resources.
7. REST services commonly exchange data in JSON format because it is lightweight and easy to understand. XML can also be used for data exchange.
8. REST supports layered architecture where different layers work independently. This improves security, scalability, and flexibility of applications.
9. Example of a REST API is an online shopping application where /products returns product details. A client application can request this data using a GET request.
10. REST architecture is simple, scalable, and platform-independent. It allows easy communication between different applications and systems over the internet.

6 Explain handling of HTTP methods (GET, POST, PUT, DELETE) in Spring Boot.

1. Spring Boot handles HTTP methods in REST APIs using mapping annotations. These annotations connect HTTP requests with controller methods.
2. The GET method is used to retrieve data from the server. In Spring Boot, @GetMapping annotation is used to handle GET requests.
3. Example: @GetMapping("/students") can return the list of all students from the database. GET requests do not modify existing data.
4. The POST method is used to create new data on the server. Spring Boot uses the @PostMapping annotation to handle POST requests.
5. Example: @PostMapping("/students") can add a new student record into the database. Data is usually sent in JSON format in the request body.
6. The PUT method is used to update existing records completely. Spring Boot handles it using the @PutMapping annotation.
7. Example: @PutMapping("/students/{id}") updates details of a student using the student ID. Existing data is replaced with updated information.
8. The DELETE method is used to remove data from the server. Spring Boot uses the @DeleteMapping annotation for delete operations.
9. Example: @DeleteMapping("/students/{id}") deletes a student record from the database using the student ID. This method permanently removes the resource.
10. These HTTP methods help create complete RESTful APIs in Spring Boot applications. They provide simple and standardized communication between client and server.

7 Develop a RESTful web service using @RestController to perform CRUD operations.

1. A RESTful web service in Spring Boot is created using the @RestController annotation. It helps in handling HTTP requests and returning responses in JSON format.
2. CRUD operations stand for Create, Read, Update, and Delete. These operations are performed using POST, GET, PUT, and DELETE HTTP methods.
3. First, create a Spring Boot project and add the Spring Web dependency. Then create a controller class to handle API requests.
4. @GetMapping is used to retrieve data, while @PostMapping is used to add new data. @PutMapping updates existing data and @DeleteMapping removes data from the server.
5. Example of a REST controller performing CRUD operations:

```
@RestController
public class StudentController {

  @GetMapping("/students")
  public String getStudents() {
    return "Student List";
  }

  @PostMapping("/students")
  public String addStudent() {
    return "Student Added";
  }

  @PutMapping("/students/{id}")
  public String updateStudent() {
    return "Student Updated";
  }

  @DeleteMapping("/students/{id}")
  public String deleteStudent() {
    return "Student Deleted";
  }
}
```

6. Spring Boot automatically handles request mapping and response conversion in REST APIs. RESTful services are widely used in web and mobile applications.

8 Explain JSON-based data exchange in REST APIs with example.

1. JSON stands for JavaScript Object Notation and is commonly used for data exchange in REST APIs. It is lightweight, simple, and easy for humans and machines to understand.
2. JSON stores data in the form of key-value pairs. It also supports objects, arrays, strings, numbers, and boolean values.
3. In REST APIs, clients send requests to the server in JSON format. The server processes the request and returns the response in JSON format.
4. JSON helps different applications communicate easily over the internet. It is supported by many programming languages such as Java, Python, JavaScript, and PHP.
5. Spring Boot automatically converts Java objects into JSON responses using built-in libraries. This reduces manual coding and simplifies REST API development.
6. Example of JSON data for a student object:

```
{
  "id": 101,
  "name": "Rahul",
  "course": "BCA"
}
```

7. JSON is preferred over XML because it requires less storage space and transfers data faster. It improves application performance and communication speed.
8. JSON-based data exchange is widely used in web applications, mobile apps, and cloud services. It plays an important role in modern RESTful web services.

9. Explain REST API testing using Postman with steps and examples.

1. Postman is a popular tool used for testing REST APIs and web services. It provides an easy interface for sending HTTP requests and viewing responses.
2. Developers use Postman to test API methods such as GET, POST, PUT, and DELETE. It helps verify whether APIs are working correctly or not.
3. The first step in API testing is opening Postman and entering the API URL. Then the required HTTP method is selected from the dropdown menu.
4. For a GET request, the user enters the API URL and clicks the Send button. The server returns the requested data as a response, usually in JSON format.
5. For a POST request, users enter JSON data in the request body before sending the request. This is commonly used to add new records to the database.
6. Example of JSON data used in Postman:

```
{
  "id": 101,
  "name": "Rahul",
  "course": "BCA"
}
```

7. Postman also displays response details such as status code, headers, and response body. These details help developers identify errors and debug APIs easily.
8. API testing using Postman improves application reliability and reduces development errors. It is widely used for testing REST APIs in Spring Boot applications.

10. Design a simple Spring Boot REST API for student management system.

1. A Student Management REST API is used to manage student records using HTTP requests. Spring Boot provides annotations and embedded servers to simplify API development.
2. First, create a Spring Boot project using Spring Initializer and add the Spring Web dependency. This dependency is required for creating REST APIs.
3. Create a Student class containing fields such as id, name, and course. This class represents the student data structure in the application.
4. Create a controller class and annotate it with @RestController. This controller handles all API requests related to students.
5. Use @GetMapping to retrieve student details and @PostMapping to add new student records. @PutMapping updates records and @DeleteMapping removes records.
6. Example of a simple REST API controller:

```
@RestController
public class StudentController {

  @GetMapping("/students")
  public String getStudents() {
    return "Student List";
  }

  @PostMapping("/students")
  public String addStudent() {
    return "Student Added";
  }

  @PutMapping("/students/{id}")
  public String updateStudent() {
    return "Student Updated";
  }

  @DeleteMapping("/students/{id}")
  public String deleteStudent() {
    return "Student Deleted";
  }
}
```